ConversAI — how it works

One laptop · one screen facing the Deaf guest · the receptionist only talks and listens

■ Intent classification (highlighted)

The receptionist speaks → the guest sees ASL

1 Speak
The receptionist holds Space and talks
Microphone

2 Transcribe
The speech becomes text
OpenAI Whisper

3 Understand
What does the receptionist mean?
Intent classifier

4 Translate
The text becomes ASL signs
ASL gloss

5 Sign
A 3D bellhop signs it, with captions
3D avatar

The guest signs → the receptionist hears English

1 Sign
The guest signs to the webcam
Camera

2 Track
Hands, body and face are tracked
MediaPipe

3 Recognize
Each sign is recognized
Our sign classifier

4 Understand
Which sentence does the guest mean?
Intent decoder

5 Speak
The answer is said out loud
ElevenLabs voice

87
signs the avatar can play, recorded by the team

1,320
training takes recorded by 5 signers

≈ 90 %
of sentences understood for a signer never seen

ConversAI — technical workflow

One laptop · one screen facing the Deaf guest · the receptionist only talks and listens

■ Intent classification (highlighted)

RECEPTIONIST → GUEST speech to ASL

1 Push-to-talk

Space held = record

- MacBook built-in mic, chosen automatically (AirPods / iPhone avoided)
- MediaRecorder → webm; room noise never transcribed

2 Speech to text

OpenAI Whisper (whisper-1)

- Next.js route /api/transcribe, key server-side
- fallback: Chrome Web Speech API

3 INTENT CLASSIFIER

20 receptionist intents

- keywords + similarity to the reference sentence (see below)
- /api/gloss → { gloss, intents }

4 Text → ASL gloss

intent → curated ASL gloss

- ASL word order, BROWS-UP / DOWN on questions, numbers
- else word by word: 87 signs, synonyms, plurals, fingerspelling

5 3D signing avatar

Three.js · rigged bellhop (GLB)

- Tripo mesh, Mixamo skeleton, 3 phalanges per finger
- replays the team's MediaPipe captures: arm IK, finger aiming, 30 fps, captions

GUEST → RECEPTIONIST ASL to speech

1 Guest turn & capture

recognition/live_key.py · OpenCV

- starts when the avatar has finished signing
- ends when the hands rest 2.5 s; live preview on the page

2 Landmarks

MediaPipe Holistic 1.1.0, on-device

- 33 pose + 2 × 21 hand + 26 face points
- shoulder-normalized, 30 fps; no video leaves the laptop

3 Sign classifier

PyTorch · logistic regression, 22 classes

- trained on the team's 1,320 takes (see below)
- scores sliding 1 s windows (step 0.13 s)

4 INTENT DECODER

22 guest sentences = guest intents

- aligns each sentence on the window probabilities (see below)
- recognition/sentence.py

5 Text to speech

ElevenLabs (eleven_flash_v2_5)

- Next.js route /api/tts; fallback: browser voice
- spoken on the laptop + “You said: …” on screen

HOW THE CLASSIFIERS ARE BUILT

Intent classifier — receptionist

Rule-based · TypeScript · no training

- 20 intents, one per receptionist sentence of docs/vocabulary.md
- keyword patterns per intent (regex), incl. Whisper variants: keycard, wi-fi, booking…
- kept only if ≥ 1/3 of the sentence's content words belong to its reference sentence; ties → most specific rule
- slots by regex: room number, times → ASL numbers
- deterministic, unit-tested: 20 / 20 vocabulary sentences

Sign classifier

Trained · PyTorch · on our own data

- data: 1,320 takes = 5 signers × 22 classes × 12, MediaPipe (capture/train_session.py)
- input: 32 steps × 141 features → 7 stats each (mean, std, min, max, 3 thirds) = 987 values, standardized
- model: multinomial logistic regression = 1 linear layer 987 → 22 (21,736 params), softmax + cross-entropy
- training: L-BFGS (300 it.), L2 λ = 0.001, temperature 1.53 fitted on a held-out signer
- beat a 2-layer BiGRU (AdamW, augmentation): 89.8 % vs 87.9 % (leave-one-signer-out)

Intent decoder — guest

Algorithm · Python / NumPy · no training

- input: classifier probabilities on sliding 1 s windows
- each of the 22 guest sentences is aligned on the windows: its signs in order, each on a contiguous run (monotonic dynamic programming)
- score = mean log-probability; the best sentence wins, so misread signs are fixed by the sentence
- window length tuned on a held-out signer: 1 s → 90 %, 1.7 s → 84 %

Turn-taking web/lib/turns.ts

FRONT DESK → GUEST → SPEAKING → FRONT DESK · mic muted during the guest turn · events via POST /api/guest/signs → SSE

Stack

Next.js 16 · React 19 · Three.js · Python 3.12 · MediaPipe 1.1.0 · OpenCV · PyTorch · Whisper · ElevenLabs

Results

89.8 % isolated signs, unseen signer (top-3 97.1 %) · ≈ 90 % continuous sentences (simulated) · 8 / 8 live, unguided

Appendix — why our own sign classifier?

We use open source where it is strong, and trained only the last layer on our hotel vocabulary

WHAT WE CHECKED

Open sign-language models

recognize isolated signs from their own vocabulary, or fingerspelling (Kaggle competitions); none translated hotel conversations reliably when we started

Google ASL dataset (Kaggle GISLR, 250 signs)

covers only 7 of our 21 signs: yes, no, please, thank you, where, time, room. Missing: reservation, elevator, towel, breakfast, checkout, password...

ASL Citizen

license forbids redistribution; non-commercial research use only

WLASL

videos scraped from third-party sites, many dead links, unclear license

WHAT WE BUILT INSTEAD

21,736 — parameters: runs in milliseconds on the laptop CPU

1,320 — takes recorded by 5 team members (22 classes × 12)

≈ 90 % — sentences right for a signer never seen

0 — video frames leave the laptop: only skeleton points, processed locally

Open source we do use

MediaPipe (Google) for hand, body and face tracking · PyTorch for training · Three.js for the avatar · Next.js for the app · Whisper speech model (via the OpenAI API)

Next step: when an open continuous ASL-to-English model is available, it can replace our classifier; tracking, sentence decoding, avatar and voice stay the same.